

Ciclo while

Ciclo while

Come ripetere istruzioni con il ciclo while in C++.

A cosa serve il ciclo `while` ?

A volte non sai in anticipo quante volte dovrai ripetere qualcosa. Per esempio: continua a chiedere all'utente un numero finché non inserisce uno valido. Oppure: elabora dati finché non hai finito la lista.

Il ciclo `while` ripete un blocco di codice **finché una condizione rimane vera**. Si ferma solo quando la condizione diventa falsa.

La struttura del ciclo `while`

```
while (condizione) {  
    // codice da ripetere  
}
```

Un esempio concreto:

```
int i = 1;  
  
while (i <= 5) {  
    cout << i << endl;  
    i++; // aumentiamo i di 1 ad ogni giro  
}
```

Output:

1
2
3
4
5

Come funziona, passo per passo:

1. Controlla la condizione `i <= 5`
2. Se è vera, esegue il blocco
3. Torna al punto 1
4. Quando la condizione è falsa, si ferma

Il contatore: non dimenticarlo!

Di solito si usa una variabile **contatore** per tenere traccia dei giri. Devi:

1. Inizializzarla **prima** del ciclo
2. Aggiornarla **dentro** il ciclo

```
int contatore = 0;

while (contatore < 3) {
    cout << "Esecuzione " << contatore + 1 << endl;
    contatore++; // senza questa riga: ciclo infinito!
}
```

Il ciclo infinito: cosa evitare

Se la condizione non diventa mai falsa, il ciclo gira per sempre. Il programma si blocca:

```
// ATTENZIONE: questo è un ciclo infinito!
int i = 1;
while (i <= 5) {
    cout << i << endl;
    // manca i++ - i rimane sempre 1, la condizione non cambia mai
}
```

Se ti capita, puoi fermare il programma premendo `Ctrl+C` nel terminale.

Quando il `while` è più adatto del `for`

Il `while` eccelle quando non sai in anticipo quante ripetizioni ci saranno. Per esempio, continuare a leggere input finché l'utente non decide di uscire:

```
int numero;
cout << "Inserisci un numero (0 per uscire): ";
cin >> numero;

while (numero != 0) {
    // calcoliamo il quadrato del numero inserito
    cout << "Il quadrato di " << numero << " è " << numero * numero <<
endl;

    cout << "Inserisci un numero (0 per uscire): ";
    cin >> numero;
}

cout << "Arrivederci!" << endl;
```

Ciclo con accumulatore

Un pattern molto comune: sommare o raccogliere valori uno per uno:

```

int somma = 0;    // accumulatore – parte da zero
int numero;
int n = 5;        // quanti numeri vogliamo leggere

cout << "Inserisci " << n << " numeri:" << endl;

int i = 0;
while (i < n) {
    cout << "Numero " << (i + 1) << ": ";
    cin >> numero;
    somma += numero; // aggiungiamo il numero alla somma totale
    i++;
}

cout << "Somma: " << somma << endl;
cout << "Media: " << (double)somma / n << endl;

```

Condizioni complesse

Puoi combinare più condizioni con `&&` e `||` :

```

int tentativi = 0;
int password;
const int PASSWORD_CORRETTA = 1234;

// Continua finché la password è sbagliata E i tentativi sono meno di 3
while (password != PASSWORD_CORRETTA && tentativi < 3) {
    cout << "Inserisci la password: ";
    cin >> password;
    tentativi++;

    if (password != PASSWORD_CORRETTA) {
        cout << "Password errata. Tentativi rimasti: " << (3 - tentativi)
<< endl;
    }
}

if (password == PASSWORD_CORRETTA) {
    cout << "Accesso consentito!" << endl;
} else {
    cout << "Accesso bloccato dopo 3 tentativi." << endl;
}

```

Esempio pratico: calcolatrice interattiva

```
#include <iostream>
using namespace std;

int main() {
    char continua = 's';

    // Continuiamo finché l'utente vuole fare calcoli
    while (continua == 's' || continua == 'S') {
        double a, b;
        char operazione;

        cout << "Inserisci il primo numero: ";
        cin >> a;
        cout << "Operazione (+, -, *, /): ";
        cin >> operazione;
        cout << "Inserisci il secondo numero: ";
        cin >> b;

        if (operazione == '+') {
            cout << "Risultato: " << a + b << endl;
        } else if (operazione == '-') {
            cout << "Risultato: " << a - b << endl;
        } else if (operazione == '*') {
            cout << "Risultato: " << a * b << endl;
        } else if (operazione == '/') {
            if (b != 0) {
                cout << "Risultato: " << a / b << endl;
            } else {
                cout << "Errore: divisione per zero!" << endl;
            }
        }

        cout << "Vuoi continuare? (s/n): ";
        cin >> continua;
    }

    cout << "Ciao!" << endl;
    return 0;
}
```